AVP32F069 数字信号处理器

勘误表

编号：JXDZ3.590.018 ZYSX

Advancechip Electronics

湖南进芯电子科技有限公司

2023 年 12 月

V1.0

1. 使用说明

序 号	描 述	影 响
1	PIE：写入 PIEACK 和手动清除 CPU 中断掩码连续操作会引起虚假的中断嵌套。	
2	CAN 引导加载程序：内部振荡器容差不足以支撑高温下 CAN 的正常运行。	
3	FPU32 和 VCU 连续内存访问。	
4	使用嵌套中断时的注意事项。	

2. 通告说明

序 号	描 述	影 响
1	FPU：CPU 到 FPU 寄存器移动操作，然后是 F32TOUI32，FRACF32 或 UI16TOF32 操作	
2	FPU：FPU 到 CPU 寄存器移动操作之前的任何 FPU 2p 操作	
3	FPU：LUF、LVF 标志对于 EINVF32 和 EISQRTF32 指令无效	
4	VCU：第一个 CRC 计算可能不正确	
5	VCU：溢出标志设置不正确	
6	CLA：内存和时钟配置（MMEMCFG）寄存器位 8、9 和 10 是只写的	
7	DMA：通过 PERINTSEL 选择 ePWM 中断触发源不正确	
8	振荡器：CPU 时钟切换到 INTC2 可能会导致复位后出现时钟缺失情况	
9	看门狗：当 WDCLK 源是 OSCCLKSRC2 时 CPU 看门狗操作不正确	
10	内存：预取超出有效内存	
11	GPIO： GPIO 限定	
12	ADC：初始转换	
13	ADC：温度传感器最小采样窗口要求	
14	ADC：采样在第 14 个周期结束时的 ADC 结果转换上一次转换，ACQPS = 6 或 7	
15	ADC：失调自重新校准要求	
16	ADC：ADC 修订寄存器（ADCREV）限制	
17	ADC：在转换期间写入 ADCNONOVERLAP 或 RESET 时，ADC 可能会变得无响应	
18	ePWM：SWFSYNC 无法正确传播到 EPWMSYNCO 引脚上的后续 ePWM 模块或输出	
19	ePWM：如果跳闸在消隐窗口结束时保持活动状态，则可能发生 ePWM 故障	
20	ePWM：消隐窗口开始后的前 3 个周期的消隐窗口不会过滤跳闸事件	
21	eQEP：错过第一个索引事件	
22	eQEP：GPIO 异步模式下的 eQEP 输入	
23	eQEP：GPIO25 引脚上的 EQEP2B 功能操作不正确（此通报仅适用于 100 引脚封装）	
24	eQEP：在指数期间方向变化时，位置计数器错误地复位	
25	eCAN：中止未设置确认位	
26	eCAN：传输操作意外停止	
27	USB：USB DMA 事件触发器导致过多的 DMA 传输	
28	USB：主机模式——无法通过集线器与低速设备通信	

1.使用说明

1.用 法	PIE：写入 PIEACK 和手动清除 CPU 中断掩码连续操作会引起虚假的中断嵌套。
影响范围	所有版本
详 述	用于中断嵌套的某些代码顺序会让 CPU 和 PIE 进入不一致的状态，从而触发不需要的中断。进入此状态所需的条件是： 1. PIEACK 清除后立即显示启用全局中断（EINT 或 asm（“ CLRC INTM”））。 2. 嵌套中断为其组清除一个或多个 PIEIER 位。 是否触发不需要的中断取决于系统中其他中断的配置和时序。在大多数应用程序中，这应该是一个罕见或不存在的事件。如果发生这种情况，不需要的中断将是嵌套中断的 PIE 组中的第一个中断，并将在嵌套中断重新启用 CPU 中断（EINT 或 asm（“ CLRC INTM”））后被触发。

解决方案	在 PIEACK 写入和 CPU 中断启用之间添加 NOP。示例代码如下所示。
	`//Bad interrupt nesting code`
	`PieCtrlRegs.PIEACK.all = 0xFFFF; //Enable nesting in the PIE`
	`EINT; //Enable nesting in the CPU`
	
	`//Good interrupt nesting code`
	`PieCtrlRegs.PIEACK.all = 0xFFFF; //Enable nesting in the PIE`
	`asm(" NOP"); //Wait for PIEACK to exit the pipeline`
	`EINT; //Enable nesting in the CPU`

2.用 法	CAN 引导加载程序：内部振荡器容差不足以支撑高温下 CAN 的正常运行。
影响范围	所有版本
详 述	设备 Boot-ROM 中的 CAN 引导加载程序使用内部振荡器作为 CAN 时钟源。在高温下，内部振荡器的频率可能会偏离较大，从而影响 CAN 消息的接收，导致通讯异常。
解决方案	在调用 CAN 引导加载程序之前重新校准内部振荡器。这可以在应用程序代码中完成。为了获得更大的灵活性，可以将包装器功能编程到器件的 OTP 存储器中。

3.用 法	FPU32 和 VCU 先后对内存访问。
影响范围	所有版本

详 述	当 VCU 内存访问和 FPU 内存访问先后发生时，此用法说明适用。有三种情况：
	情况 1：
	连续内存读取：先由 VCU 指令（VMOV32）执行一次读取，随后 FPU32 指令（MOV32）执行一次读取。如果在第一次读取期间流水线 R1 阶段发生停顿，则第二次读取将锁定错误的数据。如果第一条指令在 R1 阶段未停止，则第二次读取将正确进行。
	指令的顺序（FPU 后跟 VCU 或 VCU 后跟 FPU）无关紧要。任何一次读取访问的内存地址也无关紧要。
	情况 1 解决方法：在两个连续的读取指令之间插入一条指令。可以使用除 VCU 或 FPU 内存读取以外的任何指令。
	情况 1，示例 1：
	`VMOV32 VR1,mem32 ; VCU memory read`
	`NOP ; Not a FPU/VCU memory read`
	`MOV32 ROH,mem32 ; FPU memory read`
	情况 1，示例 2：
	`VMOV32 VR1,mem32 ; VCU memory read`
	`VMOV32 mem32, VR2 ; VCU memory write`
	`MOV32 ROH,mem32 ; FPU memory read`
	情况 2：
	连续内存写入：先由 VCU 指令（VMOV32）执行一次写入，随后 FPU32 指令（MOV32）执行一次写入。如果在第一次写入期间流水线停止，则第二次写入可能会损坏数据。如果第一条指令在写入阶段没有停止，则不会发生损坏。

	指令的顺序（FPU 后跟 VCU 或 VCU 后跟 FPU）无关紧要。任何一次写入的内存地址也无关紧要。
	情况 2 解决方法：在 VCU 和 FPU 连续写入指令之间插入两条指令。可以使用除 VCU 或 FPU 内存写入以外的任何指令。
	情况 2，示例 1：
	`VMOV32 mem32,VR0 ; VCU memory write`
	`NOP ; Not a FPU/VCU memory write`
	`NOP ; Not a FPU/VCU memory write`
	`MOV32 mem32,R3H ; FPU memory write`
	情况 2，示例 2：
	`VMOV32 mem32,VR0 ; VCU memory write`
	`VMOV32 VR1, mem32 ; VCU memory read`
	`NOP ; NOP`
	`MOV32 mem32,R3H ; FPU memory write`
	情况 3：
	连续的内存写后读，或内存读后写。在这种情况下，两条指令之间没有交互。无需执行任何操作。

解决方案	

4.用 法	中断嵌套程序应注意事项
影响范围	所有版本
详 述	如果用户在中断服务程序（ISR）中使用 EINT 指令启用中断以使用嵌套功能，则用户必须在退出 ISR 之前使用 DINT 禁用中断。如果不这样做，可能会导致 CPU 执行未定义的行为。

、2.通告说明

1.通　　告	FPU：CPU 寄存器到 FPU 寄存器 MOVE 操作后，紧跟 F32TOUI32，FRACF32 或 UI16TOF32 等操作指令。
影响范围	
描　　述	从 CPU 寄存器 FPU 寄存器的写入阶段与 F32TOUI32、FRACF32 或 UI16TOF32 指令的执行阶段一致时，此通报适用。如果 F32TOUI32 指令执行和 CPU-to-FPU 寄存器写入操作发生在同一周期内，则目标寄存器（CPU 到 FPU 寄存器写入操作）将被 F32TOUI32 指令的输出覆盖，而不是 CPU 数据写入总线上存在的数据。此方案也适用于以下说明: • F32TOUI32 RaH, RbH • FRACF32 RaH , RbH • UI16TOF32 RaH , mem16 • UI16TOF32 RaH , RbH

解决方案	在 CPU-to-FPU 寄存器写入之后必须有 5 个 NOP 的间隙或非冲突指令，然后才能使用 F32TOUI32、FRACF32 或 UI16TOF32。
	代码生成工具 v6.0.5（对于 6.0.x 分支）、v6.1.2（对于 6.1.x 分支）以及更高版本检查此方案。
	问题示例：
	`SUBF32 R5H, R3H, R1H`
	`|| MOV32 *--XAR4, R4H`
	`EISQRTF32 R4H, R2H`
	`UI16TOF32 R2H, R3H`
	`MOV32 R0H, @XAR0 ; Write to R0H register`
	`NOP ; Only 2 NOP`
	`NOP ;`
	`F32TOUI32 R1H, R1H ; R1H gets written to R0H`
	`I16TOF32 R6H, R3H`
	解决方法示例：
	`SUBF32 R5H, R3H, R1H`
	`|| MOV32 *--XAR4, R4H`
	`EISQRTF32 R4H, R2H`
	`UI16TOF32 R2H, R3H`
	`MOV32 R0H, @XAR0 ; Write to R0H register`
	`NOP ; Need 5 NOP`
	`NOP`
	`NOP`
	`NOP`
	`NOP`
	`F32TOUI32 R1H, R1H ;`
	`I16TOF32 R6H, R3H`

2.通　　告	FPU：FPU 到 CPU 寄存器移动操作之前的任何 FPU 2p 操作
影响范围	
描　　述	当多周期（2p）FPU 指令后跟 FPU 到 CPU 寄存器传输时，此通报适用。如果 FPU 到 CPU 读取指令源寄存器与 2p 指令目标相同，则在 2p 指令完成之前，读取的值可能是 FPU 寄存器的值。发生这种情况是因为 2p 指令依赖于在管道的 E3 阶段结果的前导数据。如果在 E3 阶段碰巧发生管道停顿，则不会及时转发结果以进行读取指令。 受此通报影响的 2p 指令为 MPYF32、ADDF32、SUBF32 和 MACF32.读取 FPU 寄存器的目标必须是 CPU 寄存器（ACC、P、T、XAR0…XAR7）。如果寄存器读取是 FPU 到 FPU 寄存器的传输，则此通报不适用。 在下面的示例中，2p 指令 MPYF32 使用 R6H 作为其目标。读取的 FPU 寄存器 MOV32 使用相同的寄存器 R6H 作为其源，并使用 CPU 寄存器作为目标。如果在 E3 流水线阶段发生失速，则 MOV32 将在 MPYF32 指令完成之前读取 R6H 的值。 ***问题示例：***

```
   MPYF32 R6H, R5H, R0H  ; 2p FPU instruction that writes to R6H
|| MOV32 *XAR7++, R4H
   F32TOUI16R R3H, R4H   ; delay slot
   ADDF32 R2H, R2H, R0H
|| MOV32 *--SP, R2H      ; alignment cycle
   MOV32 @XAR3, R6H      ; FPU register read of R6H
```

图 3-1 显示了管道中没有停顿时问题的管道图。

	Instruction	F1	F2	D1	D2	R1	R2	E	W		备　注
		FPU pipeline--->				R1	R2	E1	E2	E3	
I1	MPYF32 R6H, R5H, R0H || MOV32 *XAR7++, R4H	I1									
I2	F32TOUI16R R3H, R4H	I2	I1								
I3	ADDF32 R3H, R2H, R0H || MOV32 *--SP, R2H	I3	I2	I1							
I4	MOV32 @XAR3, R6H	I4	I3	I2	I1						
			I4	I3	I2	I1					
				I4	I3	I2	I1				
					I4	I3	I2	I1			
						I4	I3	I2	I1		
							I4	I3	I2	**I1**	原本 I4 进入 R2 阶段时采样结果。I1 乘积 R6H=R5H*R0H 在 E3 阶段完成计算，但作为操作数转发到 I4。这使得 I4 看起来像是 2p 指令，但 I4 实际上需要 3p 周期来计算。
								I4	I3	I2	
									I4	I3	

图 3-1 管道中没有停顿时的问题图

图 3-2 显示了如果指令 I1 的 E3 插槽中出现停顿，则问题的管道图

	Instruction	F1	F2	D1	D2	R1	R2	E	W		备　注
		FPU pipeline--->				R1	R2	E1	E2	E3	
I1	MPYF32 R6H, R5H, R0H || MOV32 *XAR7++, R4H	I1									
I2	F32TOUI16R R3H, R4H	I2	I1								
I3	ADDF32 R3H, R2H, R0H || MOV32 *--SP, R2H	I3	I2	I1							
I4	MOV32 @XAR3, R6H	I4	I3	I2	I1						
			I4	I3	I2	I1					
				I4	I3	I2	I1				
					I4	I3	I2	I1			

						I4	I3	I2	I1		
							I4	I3	I2	**I1** (STALL)	
								I4	I3	I2	
									I4	I3	

解决方案

在此方案中，将 MPYF32、ADDF32、SUBF32 和 MACF32 视为 3p 周期指令。必须将三个 NOP 或非冲突指令放置在指令的延迟插槽中。

代码生成工具 v.6.2.0 及更高版本将生成正确的指令序列并检测汇编代码中的错误。在以前的版本 v6.0.5（对于 6.0.x 分支）和 v.6.1.2（对于 6.1.x 分支）中，编译器将生成正确的指令序列，但汇编程序不会检测汇编代码中的错误。

解决方法示例：

```
   MPYF32 R6H, R5H, R0H  ; 2p FPU instruction that writes to R6H
|| MOV32 *XAR7++, R4H
   F32TOUI16R R3H, R4H   ; delay slot
   ADDF32 R2H, R2H, R0H
|| MOV32 *--SP, R2H      ; delay slot
   NOP                   ; alignment cycle
   MOV32 @XAR3, R6H      ; FPU register read of R6H
```

	Instruction	F1	F2	D1	D2	R1	R2	E	W		备 注
		FPU pipeline--->				R1	R2	E1	E2	E3	
I1	MPYF32 R6H, R5H, R0H || MOV32 *XAR7++, R4H	I1									
I2	F32TOUI16R R3H, R4H	I2	I1								
I3	ADDF32 R3H, R2H, R0H || MOV32 *--SP, R2H	I3	I2	I1							
I4	NOP	I4	I3	I2	I1						
I5	MOV32 @XAR3, R6H	I5	I4	I3	I2	I1					
			I5	I4	I3	I2	I1				
				I5	I4	I3	I2	I1			
					I5	I4	I3	I2	I1		
						I5	**I4**	I3	I2	**I1** (STALL)	
							I5	I4	I3	I2	
								I5	I4	I3	

3.通　　告	FPU：LUF、LVF 标志对于 EINVF32 和 EISQRTF32 指令无效。
影响范围	
描　　述	此通报适用于 EINVF32 和 EISQRTF32 指令。这些说明的预期结果是正确的;但是，下溢 （LUF） 和溢出 （LVF） 标志不是。这些标志无效，不应使用。 LUF 和 LVF 标志无法使用 C 代码访问，因此此通报的总体影响预计很小。如果用户选择在程序集中将这些标志（例如，在对时间关键型算法进行编码时）作为混合 C/ASM 项目的一部分，则用户将需要使用这些标志禁用围绕程序集代码的中断，并且还需要通过使用 EINVF32 或 EISQRTF32 指令来保留这些标志。
解决方案	在 C 代码中使用这些标志没有解决方法，并且由于编译器和工具用法说明中提出的原因，应将其视为无效。 此处显示的解决方法提供了一种在仅程序集代码中使用 EISQRTF32 和 EINVF32 时保留 LVF、LUF 标志的方法。 不要依赖 LUF 和 LVF 标志来捕获由 EINVF32 和 EISQRTF32 指令引起的下溢/溢出情况。相反，在使用每条指令之前，请检查操作数的以下条件（在代码中）： EINVF32　　Divide by 0 EISQRTF32　　Divide by 0, Divide by a negative input 通过在调用指令之前将标志保存到堆栈中，然后一旦指令完成，然后恢复标志的值，忽略 LUF 和 LVF 标志的内容。 如果 PIE 中断绑定到 LUF 和 LVF 标志，请在使用 EINVF32 或 EISQRTF32 指令之前禁用中断（在 PIE 上）。检查是否设置了 LUF 和 LVF 标志;如果是，则可以设置一个变量来指示检测到错误的 LUF/LVF 条件。在重新启用中断之前，请清除 STF（FPU 状态标志）中的标志。 一旦在 PIE 上重新启用中断，就可能发生中断（如果两条指令中的任何一条断言了 LUF/LVF 中断线路），并执行分支到中断服务例程 （ISR）。检查标志以确定是否发生了错误情况;如果有，请忽略中断。 不要直接清除 PIEIFR 位（锁定 LUF 和 LVF 标志），因为同一 PIE 组（PIE 组 12）上的中断事件可能会无意中丢失。 下面是一个示例:

```
_flag_LVFLUF_set    .usect ".ebss",2,1,1
    ...
    MOV32   *SP++,STF                       ; Save off current status flags
                                            ; Load the PieCtrlRegs page to the DP
    MOVW    DP, #_PieCtrlRegs.PIEIER12.all
                            ; Zero out PIEIER12.7/8, i.e. disable LUF/LVF interrupts
    AND     @_PieCtrlRegs.PIEIER12.all, #0xFF3F
    EISQRTF32/EINVF32                       ; Execute operation
    MOVL    XAR3, #_flag_LVFLUF_set         ; Wait for operation to complete
    MOV32   *+XAR3[0], STF                  ; save STF to _flag_LVFLUF_set
    AND     *+XAR3[0], #0x3                 ; mask everything but LUF/LVF
                                            ; Clear Latched overflow, underflow flag
    SETFLG  LUF=0, LVF=0
                          ; Re-enable PIEIER12.7/8, i.e. re-enable the LUF/LVF
interrupts
    OR      @_PieCtrlRegs.PIEIER12.all, #0x00C0
    MOV32   STF,*--SP                        ; Restore previous status flags
```

在 ISR 中

```
__interrupt void fpu32_luf_lvf_isr (void)
{
  // Check the flag for whether the LUF, LVF flags set by
  // either EISRTF32 or EINVF32
  if((flag_LVFLUF_set & 0x3U) != 0U)
  {
    //Reset flag
    flag_LVFLUF_set = 0U;
    // Do Nothing
  }
  else
  {
    // If flag_LVFLUF_set was not set then this interrupt
    // is the legitimate result of an overflow/underflow
    // from an FPU operation (not EISQRTF32/EINVF32)     ...
    // Handle Overflow/Underflow condition     ...
```

```
    ...
    ...
  }
  // Ack the interrupt and exit
}
```

有关编译器和工具用法的说明

编译器不使用 LVF/LUF 作为条件指令的条件代码，运行时支持（RTS）库也不以任何方式测试 LVF/LUF。

编译器可能会生成修改 LVF/LUF 的代码，这意味着 STF 寄存器（包含这些标志）的值在函数边界处未定义。因此，尽管库中的 sqrt 例程可能会导致设置 LVF/LUF，但在 CGT 中不能保证用户可以在 sqrt 返回后读取这些位。

尽管编译器确实提供了__eisqrtf 和__einvf32 内部函数，但它不提供读取 LVF/LUF 位或 STF 寄存器的内联函数。因此，用户无法从 C 代码访问这些位。

使用内联汇编代码读取 STF 寄存器是不可靠的，因此不鼓励这样做。“替代方法”部分中介绍的解决方法适用于使用 EISQRTF32 和 EINVF32 指令且不调用任何编译器生成的代码的程序集代码。对于 C 代码，用户必须认为这些标志不可靠，因此，既不在代码中轮询这些标志，也不触发它们的中断。

4.通　　告	VCU：第一个 CRC 计算可能不正确。
影响范围	
描　　述	由于 VCU 模块的内部上电状态，第一个 CRC 结果可能不正确。此条件适用于八条 CRC 指令中每条指令的第一个结果。这种罕见的情况只能在上电复位后发生，但不一定在每次上电时发生。热重置不会导致此情况再次出现。

解决方案	该应用程序可以通过在初始化例程中对单个字节执行 CRC 计算来复位内部 VCU CRC 逻辑。此例程只需要执行一个 CRC 计算，并且可以使用任何 CRC 指令。在此例程结束时，清除 VCU CRC 结果寄存器以丢弃结果。
	示例如下所示:
	`_VCUcrc_reset`
	`MOVZ XAR7, #0`
	`VCRC8L_1 *XAR7`
	`VCRCCLR`
	`LRETR`

5.通　　告	VCU：溢出标志设置不正确。
影响范围	
描　　述	
解决方案	

6.通　　告	CLA：内存和时钟配置 （MMEMCFG） 寄存器位 8、9 和 10 是只写的
影响范围	
描　　述	CPU 对 CLA 模块中 MMEMCFG 寄存器的第 8、9 和 10 位的读取将始终返回零。写入这些位将按预期工作.

解决方案	没有。要修改此寄存器的位，应使用完整配置对整个寄存器执行一次写入操作。不应使用读-修改-写入，因为对寄存器的任何读-修改-写入操作都将读取位 8、9 和 10 的零，并且可以将这些位写回零，从而无意中修改这些位。示例如下所示:
	`#define CLA_PROG_ENABLE 0x0001`
	`#define CLARAM0_ENABLE 0x0010`
	`#define CLARAM1_ENABLE 0x0020`
	`#define CLARAM2_ENABLE 0x0040`
	`#define CLA_RAM0CPUE 0x0100`
	`#define CLA_RAM1CPUE 0x0200`
	`#define CLA_RAM1CPUE 0x0400`
	`Cla1Regs.MMEMCFG.all = CLA_PROG_ENABLE1| CLARAM0_ENABLE|CLARAM1_ENABLE|`
	`CLARAM2_ENABLE|CLA_RAM1CPUE;`

7.通　　告	DMA：通过 PERINTSEL 选择 ePWM 中断触发源不正确
影响范围	
描　　述	模式。CHx[PERINTSEL] 场位值为 18–29 应选择 ePWM1SOCA– ePWM6SOCB 作为 DMA 触发源。相反，PERINTSEL 值为 18–29，选择 ePWM2SOCA–ePWM7SOCB 作为 DMA 触发源，如下表 3-1 所示。 ePWM1SOCA 和 ePWM1SOCB 不是作为 PERINTSEL 触发源实现的。
解决方案	

8.通　　告	振荡器：CPU 时钟切换到 INTC2 可能会导致复位后出现时钟缺失情况
影响范围	
描　　述	在至少两次系统复位（不包括上电复位）后，当应用程序代码尝试将 CPU 时钟源切换到内部振荡器 2 时，将发生时钟缺失情况，并且在以下情况下时钟切换将失败: • X1 和 X2 未使用（未使用时 X1 始终处于低电平）. • GPIO38（使用 TCK 和 XCLKIN 进行多路复用）仅用作 JTAG TCK 引脚. • JTAG 调试探针已断开连接。 仅当发生故障情况时，只有在上电复位后，时钟丢失情况才会恢复。

解决方案	在通过 CLKCTL 寄存器中的 OSCCLKSRCSEL 和 OSCCLKSRC2SEL 位将 CPU 时钟源切换到 INTOSC2 之前，用户必须切换 CLKCTL 寄存器中的 XCLKINOFF 和 XTALOSCOFF 位，如下序列所示:
	执行上述过程后，即可配置 OSC2 选择开关.
	
	`CLKCTL |= 0x6000;  // XCLKINOFF = 1, XTALOSCOFF = 1`
	`CLKCTL &=~0x6000;  // XCLKINOFF = 0, XTALOSCOFF = 0`
	`CLKCTL |= 0x6000;  // XCLKINOFF = 1, XTALOSCOFF = 1`
	`CLKCTL &=~0x6000;  // XCLKINOFF = 0, XTALOSCOFF = 0`
	`CLKCTL |= 0x6000;  // XCLKINOFF = 1, XTALOSCOFF = 1`
	
	如果连接了 JTAG 调试探针，并且 GPIO38（TCK）正在切换，则上述过程是不必要的，但不会造成任何伤害. 如果没有对 GPIO38 施加时钟，TI 还建议在 VDDIO 中添加 GPIO38 上的强上拉电阻.

9.通　　告	看门狗：当 WDCLK 源是 OSCCLKSRC2 时 CPU 看门狗操作不正确
影响范围	
描　　述	当 OSCCLKSRC2 用作 CPU 看门狗的时钟源时，看门狗可能会无法间歇性地生成设备复位。
解决方案	WDCLK 应仅从 OSCCLKSRC1 （INTOSC1） 获取。 CPU 可能来自 OSCCLKSRC2 或 OSCCLKSRC1 （INTOSC1）。

10.通　　告	内存：预取超出有效内存
影响范围	
描　　述	CPU 预取指令超出其管道中当前处于活动状态的指令。如果预取发生在有效内存的末尾，则 CPU 可能会收到无效的操作码。
解决方案	预取队列的深度为 8 x 16 个字。因此，代码不应出现在有效内存结束的 8 个字以内。此限制适用于设备上的所有内存区域和所有内存类型（闪存、OTP、SARAM）。跨越两个有效内存块之间的边界进行预取是可以的。 示例 1：M1 在地址 0x7FF 处结束，后面没有另一个内存块。M1 中的代码应存储不超过地址 0x7F7。0x7F8~0x7FF 地址不应用于代码。 示例 2：M0 在地址 0x3FF 结束，有效内存（M1）紧随其后。M0 中的代码可以存储到地址 0x3FF（包括地址）。代码还可以交叉到 M1，直到地址 0x7F7（包括地址）。

11.通　　告	GPIO：GPIO 限定
影响范围	
描　　述	如果将 GPIO 引脚配置为“n”SYSCLKOUT 周期鉴定期（其中 1≤n≤510），并且具有“m”鉴定样本（m = 3 或 6），则输入脉冲宽度为[n * m – （n – 1）]可能会获得合格（而不是 n * m）。这种不正确行为的发生取决于异步 GPIO 输入信号相对于内部预定标时钟相位的对准，因此不是确定性的。这种错误限定发生的概率是“1/n”。 **最坏情况示例：** 如果 n = 510， m = 6，则需要 GPIO 输入宽度 （n * m） = 3060 个 SYSCLKOUT 周期才能通过资格认证。但是，由于此通报中描述的问题，可能符合条件的最小 GPIO 输入宽度为 [n * m – （n – 1）] = 3060 – 509 = 2551 个 SYSCLKOUT 周期。

解决方案	没有。确保设计中有足够的裕量用于输入限定。

12.通　　告	**ADC：初始转换**
影响范围	
描　　述	当 ADC 转换由任何触发源在顺序或同步采样模式下启动时，第一个采样可能不是正确的转换结果。
解决方案	对于顺序模式，在每个转换系列开始时丢弃第一个样本。例如，如果应用程序要求定期启动给定的一系列转换 SOC0→SOC1→SOC2，则将序列设置为 SOC0→SOC1→SOC2→SOC3，并且仅使用最后三个转换 ADCRESULT1、ADCRESULT2、ADCRESULT3，从而丢弃 ADCRESULT0。对于同步采样模式，在每个系列转换开始时丢弃 A 和 B 通道的第一个采样。用户应用程序应验证此变通办法在其应用程序中是否可接受。 通过在 ADCCTRL2 寄存器中向 ADCNONOVERLAP 位写入 1，误差幅度显著降低，该位仅在 ADC 完成任何待处理转换时对 ADC 通道进行采样。通常，启用 ADCNONOVERLAP 后，第一个样本和后续样本之间的差异将小于或等于 4 个 LSB。

13.通　　告	ADC：温度传感器最小采样窗口要求
影响范围	
描　　述	如果使用最小采样窗口（45MHz，155.56ns 时为 6 个 ADC 时钟），则温度传感器转换的结果可能会产生较大的误差，使其对系统不可靠。
解决方案	1. 如果使用温度传感器的双重采样以避免第一个采样损坏的问题，则温度传感器结果有效。双采样等效于给采样保持电路足够的充电时间。 2. 其他任何情况下，用于对温度传感器进行采样的采样保持窗口不应小于 550ns。

14.通　　告	ADC：采样在第 14 个周期结束时的 ADC 结果转换上一次转换，ACQPS=6 或 7
影响范围	
描　　述	采样阶段结束后，片内 ADC 需要 13 个 ADC 时钟周期才能完成转换。然后在采样后的第 14 个周期将结果呈现给 CPU，并在第 15 个周期锁定到 ADC 结果寄存器中。如果下一次转换的采样阶段在第 14 个周期终止，则 CPU 锁定到结果寄存器中的结果不能保证在所有工作条件下都有效。
解决方案	一些解决方法如下: • 由于 ADC 采样和转换阶段的性质，只有两个 ACQPS 值（控制采样窗口）会导致上述情况发生（ACQPS = 6 或 7）。一种解决方案是避免在 ACQPS 中使用这些值。 • 当使用 ADCNONOVERLAP 功能（ADCTRL2 寄存器中的位 1）时，将永远不会满足上述条件;因此，用户可以自由使用所需的 ACQPS 的任何值。 • 根据系统中使用的 ADC 采样频率，用户可以确定如果系统需要使用 ACQPS = 6 或 7，他们的系统是否会达到上述条件。例如，如果转换器在 ACQPS = 6 的情况下连续转换，则永远不会满足上述条件，因为采样阶段的结束将始终落在正在进行的电流转换的第 13 个周期。

15.通　　告	**ADC：失调自重新校准要求**
影响范围	
描　　述	Device_cal（）的工厂失调校准可能无法确保 ADC 失调在客户系统的所有工作条件下保持在规格范围内。
解决方案	• 为确保失调保持在数据手册的“单次重新校准”规格范围内，请在 Device_cal（）完成且 ADC 配置完成后执行 AdcOffsetSelfCal（）功能。 • 为确保失调保持在数据手册的“定期重新校准”规格范围内，请定期执行 AdcOffsetSelfCal（）功能，以控制温度漂移。

16.通　　告	ADC：ADC 修订寄存器（ADCREV）限制
影响范围	
描　　述	ADC 修订寄存器的实现是为了区分 ADC 修订和 ADC 类型，两个字段将始终读作“0”。
解决方案	在 CLASSID（地址 0x882）为 0x009F，0x008F，0x007F 或 0x006F 的设备上，ADCREV 寄存器中的“TYPE”字段应假定为 3，“REV”字段应从下表中推断出来。

17.通　　告	ADC：在转换期间写入 ADCNONOVERLAP 或 RESET 时，ADC 可能会变得无响应
影响范围	
描　　述	当 ADCCTL2[ADCNONOVERLAP]处于在转换过程中进行了修改。在这种情况下，如果没有器件复位，ADC 将无法进行进一步转换。 有两种不同的方法会导致这种情况： 1. 在转换过程中写入 ADCCTL2[ADCNONOVERLAP]。 2. 在转换过程中写入 ADCCTL1[RESET]。
解决方案	按照以下顺序修改 ADCCTL2[ADCNONOVERLAP]或编写 ADCCTL1[RESET]： 1. 将所有 SOC 触发源 ADCSOCxCTL[TRIGSEL] 设置为 0。 2. 将所有 ADCINTSOCSEL1/2 设置为 0。 3. 确保没有其他 SOC 挂起（这可以通过轮询 SOC 标志来完成）。 4. 等待所有转换完成。 1. ADCCTL2[CLKDIV2EN] = 0， ADCCTL2[CLKDIV4EN] = x → （ACQPS + 13 ） * 1. 系统圈 2. ADCCTL2[CLKDIV2EN] = 1， ADCCTL2[CLKDIV4EN] = 0 → （ACQPS + 13 ） * 1. 系统圈 3. ADCCTL2[CLKDIV2EN] = 1， ADCCTL2[CLKDIV4EN] = 1 → （ACQPS + 13 ） *4 个系统集成电路 修改 ADCCTL2[ADCNONOVERLAP]或写入 ADCCTL1[RESET]。

```
EALLOW;
// Set all SOC trigger sources to software
AdcRegs.ADCSOC0CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC1CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC2CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC3CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC4CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC5CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC6CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC7CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC8CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC9CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC10CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC11CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC12CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC13CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC14CTL.bit. TRIGSEL  = 0;
AdcRegs.ADCSOC15CTL.bit. TRIGSEL  = 0; // Set all ADCINTSOCSEL1/2 to 0.
    AdcRegs.ADCINTSOCSEL1.bit.SOC0  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC1  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC2  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC3  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC4  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC5  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC6  = 0;
    AdcRegs.ADCINTSOCSEL1.bit.SOC7  = 0;
```

```
    AdcRegs.ADCINTSOCSEL2.bit.SOC8  = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC9  = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC10 = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC11 = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC12 = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC13 = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC14 = 0;
    AdcRegs.ADCINTSOCSEL2.bit.SOC15 = 0;
// Ensure there is not another SOC pending while (AdcRegs.ADCSOCFLG1.all !=
0x0);
// Wait for conversions to complete
// Delay time based on ACQPS = 6 , ADCCTL2[CLKDIV2EN] = 1, ADCCTL2[CLKDIV4EN]
=
0
// 7 + 13 ADC Clocks = 20 ADCCLKS -> 40 SYSCLKS
asm(" RPT#40||NOP");
// ADCCTL2[ADCNONOVERLAP] = <new value>;
// ADCCTL1[RESET] = 1;
EDIS;
```

18.通　　告	ePWM：SWFSYNC 无法正确传播到 EPWMSYNCO 引脚上的后续 ePWM 模块或输出
影响范围	
描　　述	当使用 TBCTL[SWFSYNC] 位生成软件同步脉冲时，同步信号不会向下传播到后续 ePWM 模块的 EPWMSYNCI 输入。对于 ePWM1，软件同步也不会在 EPWMSYNCO 引脚上产生输出。当通过 EPWMSYNCI 引脚上的外部信号产生同步脉冲时，同步信号在 ePWM 模块之间传播到 EPWMSYNCO 时按预期工作。
解决方案	如果应用程序需要生成软件同步： 1. 应用程序代码应为 EPWMSYNCI 操作配置单个 GPIO 多路复用器设置。 2. EPWMSYNCI 引脚应接地或始终以低电平驱动。 3. 完成上述操作后，应用程序可以使用 TBCTL[SWFSYNC] 位来生成软件同步脉冲，该脉冲传递到后续的 ePWM 模块并连接到 EPWMSYNCO 引脚。

19.通　　告	ePWM：如果跳闸在消隐窗口结束时保持活动状态，则可能发生 ePWM 故障
影响范围	
描　　述	消隐窗口通常用于屏蔽转换期间的任何 PWM 跳闸事件，这些事件将是系统的错误跳闸。如果 ePWM 跳闸事件在消隐窗口周期结束后保持活动状态的时间少于三个 ePWM 时钟，则 ePWM 输出端可能存在意外故障. 图 3-4 显示了可能导致意外 ePWM 输出的时间段。 Figure 3-4. Undesired Trip Event and Blanking Window Expiration 图 3-5 显示了如果跳闸事件在消隐窗口关闭前 1 个周期内或 3 个周期内结束，则两个潜在的 ePWM 输出可能。 ePWM Output Pin (Possibility A) (Trip interrupt will be issued) ePWM Output Pin (Possibility B) (Trip interrupt will not be issued) Figure 3-5. Resulting Undesired ePWM Outputs Possible
解决方案	延长或减少消隐窗口，以避免任何意外的跳闸操作.

20.通　　告	ePWM：消隐窗口开始后的前 3 个周期的消隐窗口不会过滤跳闸事件
影响范围	
描　　述	消隐窗口不会在开始后的前 3 个周期内将行程事件清空 消隐窗口。DCEVTFILT 可以继续反映 DCxEVTy 信号的变化。如果启用了 DCEVTFILT，则可能会影响配置的后续子系统（例如，跳变区子模块、TZ 中断、ADC SOC 或 PWM 输出）。

解决方案	在需要消隐之前启动消隐窗口 3 个周期。如果在周期边界处需要消隐窗口，请在下一个周期开始之前启动消隐窗口 3 个周期。这有效，因为空白窗口跨周期边界持续存在。

21.通　　告	eQEP：错过第一个索引事件
影响范围	
描　　述	如果 QEPI 输入端的第一个索引事件边沿发生在从相应 QEPA/QEPB 边沿之前的一个系统时钟周期到相应 QEPA/QEP 边沿之后的两个系统时钟周期的任何时间，则 eQEP 模块可能会错过此索引事件。这种情况可能会导致以下行为： 如果 QEPCTL[PCRM] = 00b 或 10b（索引事件上的位置计数器复位或第一个索引事件上的位置计数器复位），则 QPOSCNT 将不会在第一个索引事件上重置。 将不会设置第一个索引事件标记标志 （QEPSTS[FIMF]）。
解决方案	通过延迟索引信号来实现可靠操作，使得 QEPI 事件边沿在相应的 QEPA/QEPB 信号边沿之后至少发生两个系统时钟周期。对于编码器可能相对于相应的 QEPA/QEPB 信号向 QEPI 信号施加负延迟（t d）的情况（即 QEPI 边沿发生在相应的 QEPA/QEPB 边沿之前），QEPI 信号的延迟量应大于“td + 2 *SYSCLKOUT”。

22.通　　告	eQEP：GPIO 异步模式下的 eQEP 输入
影响范围	
描　　述	如果任何 eQEP 输入引脚通过 GPxQSELn 寄存器配置为 GPIO 异步输入模式，则 eQEP 模块可能无法正常工作，因为 eQEP 外设假定在模块输入上存在与 SYSCLKOUT 的外部同步。例如，QPOSCNT 可能无法正确复位或锁存，并且输入引脚上的脉冲可能会丢失。 为了 eQEP 模块的正常工作，输入 GPIO 引脚应通过 GPxQSELn 寄存器配置为同步输入模式（带或不带限定），这是 GPxQSEL 寄存器在复位时的默认状态。TI 提供的所有现有 eQEP 外设示例还将 GPIO 输入配置为同步输入模式。 异步模式不应用于 eQEP 模块输入引脚。
解决方案	配置配置为非异步模式的 eQEP 引脚的 GPIO 输入（除“11b = 异步”之外的任何 GPxQSELn 寄存器选项）。

23.通　　告	eQEP：GPIO25 引脚上的 EQEP2B 功能操作不正确（此通报仅适用于 100 引脚封装。
影响范围	
描　　述	当 GPIO25 引脚配置为 EQEP2B 功能时，无论为 MFSXA 操作配置了哪个 GPIO 引脚，MFSXA 引脚上的活动都将反映在此引脚上。仅当 GPIO25 引脚配置为 EQEP2B 操作时，才会出现此问题。当 GPIO25 引脚配置为 GPIO、ECAP2 或 SPISOMIB 操作时，不会出现此问题。
解决方案	使用 GPIO55 进行 EQEP2B 操作。

24.通　　告	eQEP：在指数期间方向变化时，位置计数器错误地复位
影响范围	
描　　述	使用 PCRM = 0 配置时，如果在索引输入处于活动状态时发生方向更改，则位置计数器 （QPOSCNT） 可能会被错误地重置，从而导致计数器值发生意外变化。这可能导致从持仓计数器的预期值更改多达 ±4 个计数，并导致错误标志的意外后续设置。 使用 PCRM = 0 配置 [即，索引事件时的位置计数器复位 （QEPCTL[PCRM] = 00）]时，如果在向前移动期间发生索引事件，则在下一个 eQEP 时钟上将位置计数器重置为 0。如果索引事件发生在反向移动期间，则位置计数器将在下一个 eQEP 时钟上重置为 QPOSMAX 寄存器中的值。eQEP 外设记录 QEPSTS 寄存器中第一索引事件标记（QEPSTS[FIDF]）上第一索引标记（QEPSTS[FIDF]）的出现和方向。它还会记住第一个索引标记上的正交边沿，以便将相同的相对正交转换用于索引事件重置操作。 如果在索引脉冲处于活动状态时发生方向变化，模块仍将继续寻找相对正交转换以执行位置计数器复位。这会导致仓位计数器值发生意外变化。
解决方案	如果在索引处于活动状态时可能发生方向更改，并且位置计数器值的由此变化可能会影响应用程序，则不要使用 PCRM = 0 配置。 用于执行位置计数器重置的其他选项（如果适用于应用程序）[如索引事件初始化 （IEI）]）没有此问题。

25.通　　告	eCAN：中止未设置确认位
影响范围	
描　　述	将传输请求重置（TRR）寄存器位设置为中止消息后，在极少数情况下，TRRn 和 TRSn 位将在不设置中止确认（AAn）位的情况下清除。传输本身已正确中止，但不会断言中断，也不会指示挂起的操作。 为了发生这种罕见的情况，必须发生以下所有情况： ● 上一条消息未成功，原因可能是仲裁丢失，要么是总线上没有节点能够确认它，要么是传输导致错误帧。上一封邮件不必来自当前尝试传输中止的同一邮箱。 ● 邮箱的 TRRn 位应设置在紧随设置 TRSn 位的周期之后的 CPU 周期中。由于传输不完整而设置的 TRSn 位也满足此条件;也就是说，TRSn 位可以在过去设置，但传输仍然不完整。 ● TRRn 位必须设置在精确的 SYSCLKOUT 周期中，其中 CAN 模块处于空闲状态一个周期。CAN 模块在不接收或传输数据的过程中，称为空闲状态。 如果发生这种情况，则在设置 TRR 位后，邮箱的 TRRn 和 TRSn 位将清除 tclr SYSCLKOUT 周期，其中： tclr = [(mailbox_number) * 2] + 3 SYSCLKOUT cycles 如果发生这种情况，则不会设置 TAn 和 AAn 位。通常，TA 或 AA 位在 TRR 位变为零之后设置。
解决方案	发生此问题时，TRRn 和 TRSn 位将在 tclr SYSCLKOUT 中清除 周期。要检查此情况，请首先禁用中断。在设置 TRRn 位后，检查 TRRn 位 tclr SYSCLKOUT 周期，以确保它仍然设置。设置 TRRn 位表示问题未发生。 如果 TRRn 位被清除，则可能是因为消息的正常端，并且设置了相应的 TAn 或 AAn 位。检查 TAn 和 AAn 位。如果设置了其中任何一个位，则问题未发生。如果它们都为零，则问题确实发生了。像中断服务例程一样处理这种情况，只是 AAn 位现在不需要清除。 如果设置了 TAn 或 AAn 位，则在重新启用中断时将发生正常的中断例程。

26.通　　告	eCAN：传输操作意外停止
影响范围	
描　　述	在极少数情况下，已经观察到来自 eCAN 模块的消息传输停止（同时接收操作正常继续）。这种异常状态可能会发生，总线上没有任何错误帧。
解决方案	可以使用 eCAN 模块的超时功能（MOTO）来检测此情况。发生这种情况时，请设置并清除 CCR 位（使用 CCE 位进行验证）以消除异常情况。

27.通　　告	USB：USB DMA 事件触发器导致过多的 DMA 传输
影响范围	
描　　述	USB 模块无意中生成额外的 DMA 请求，导致 FIFO 溢出（在 IN 端点上）或下溢（在 OUT 端点上）。这会导致无效的 IN 数据包（大于最大数据包大小）和重复的接收数据。
解决方案	使用软件 DMA 触发而不是 USB 外设请求来启动 DMA 传输。要在软件中启动 DMA 传输，请设置 CONTROL。CHx[PERINTFRC]位。

28.通　　告	USB：主机模式——无法通过集线器与低速设备通信
影响范围	
详　　情	当 USB 控制器作为主机运行并且低速数据包通过集线器发送到设备时，后续的帧启动已损坏。一段时间后，此损坏会导致 USB 控制器与集线器失去同步，从而导致数据损坏。
解决方案	没有

联系方式

公司网址： **www.advancechip.com**

联系邮箱： **sales@advancechip.com**

销售联系电话： **0731-88731027**（长沙）

公司总部地址： 长沙市湘江新区东方红街道北斗产业园·黄金园 **A5** [illegible]

南京销售中心： 南京市秦淮区卡子门大街 **19** 号紫云智慧广场 **6** 号楼 **15** 层